Shift Linked List

AlgoExpert · Hard · Linked Lists

The problem

- Rotate the list k positions forward (k > 0) or backward (k < 0); return the new head.

Intuition

- Rotation is just choosing a new tail: connect the old tail to the head and cut at length minus k.

Brute force vs optimal

- Shift one node at a time k times → O(n·k)
- Length + one cut → O(n)
- Negative k: floor mod

Approach

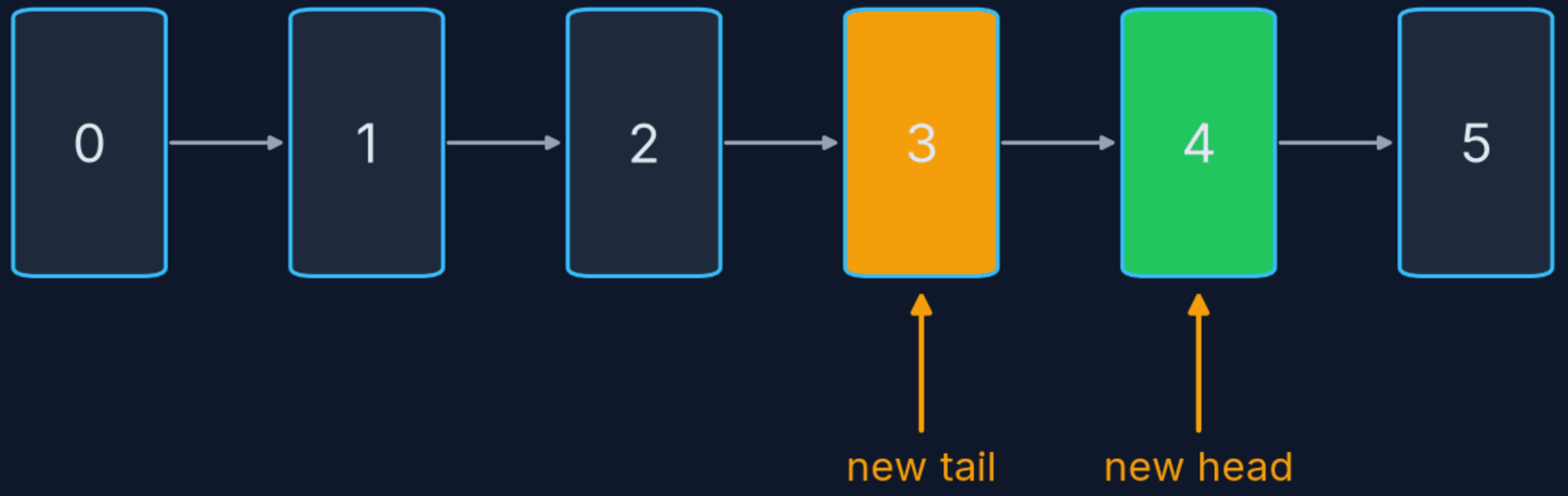
- 1. Find length and tail
- 2. Normalise k modulo length
- 3. Walk to the new tail
- 4. Rewire and return the new head

Worked example



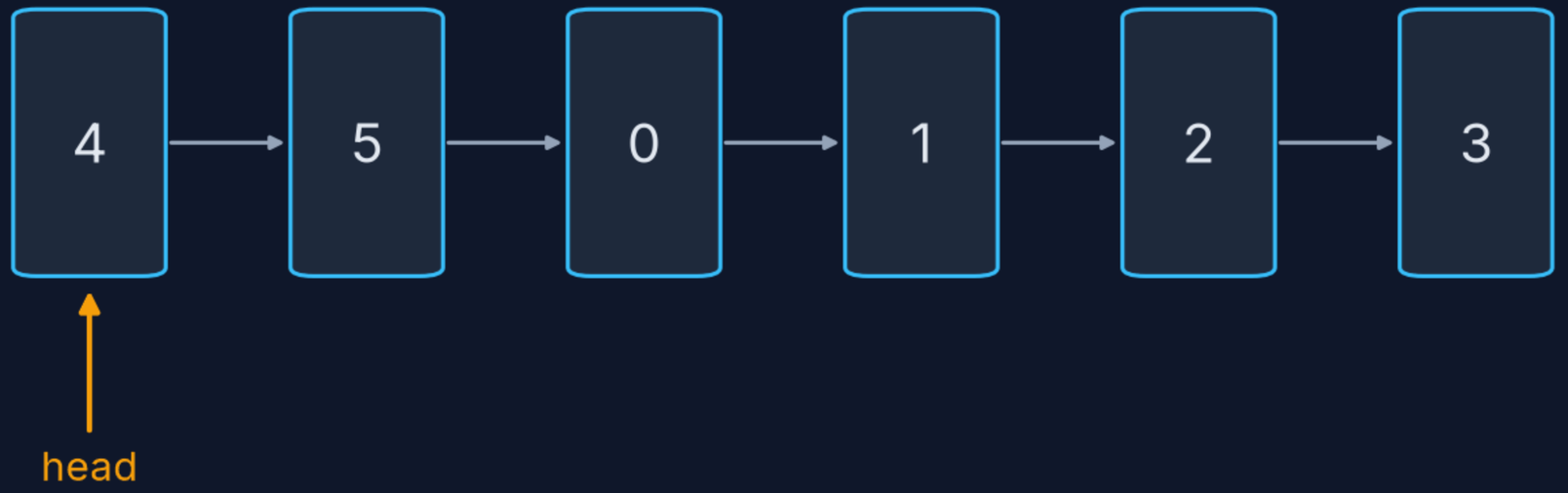
Length 6; shift 2 → effective 2

Worked example



New tail at index 3

Worked example



Cut and reconnect

Java solution

```java
public class ShiftLinkedList {
    public static LinkedListNode shiftLinkedList(LinkedListNode head, int k) {
        int len = 1;
        LinkedListNode tail = head;
        while (tail.next != null) { tail = tail.next; len++; }
        int shift = Math.floorMod(k, len);
        if (shift == 0) return head;
        LinkedListNode newTail = head;
        for (int i = 1; i < len - shift; i++) newTail = newTail.next;
        LinkedListNode newHead = newTail.next;
        newTail.next = null;
        tail.next = head;
        return newHead;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- K ≥ length → mod
- Negative k shifts backward
- Shift 0 → same list

Interview tips & follow-ups

- Same as Rotate List

Recap

- Rotation is just choosing a new tail: connect the old tail to the head and cut at length minus k.
- Complexity: O(n)